

TypeFERENCE

An authoring and reuse layer for organizational Copilot plugins

TypeFERENCE contributors - October 2026

Abstract

Organizations adopting GitHub Copilot quickly accumulate skills and custom agents that are good, local, and copied. A skill tuned in one repository is copied into the next, a team's version of a shared workflow drifts from everyone else's, and an internal marketplace has no way to stop two teams shipping different skills under one name. TypeFence treats the organization's Copilot know-how as source. Shared templates declare typed parameters; each team supplies a small data document of a declared context type; extensions specialize shared skills without copying them; MCP servers are declared once and travel with the skills that need them. A deterministic compiler resolves all of it into ordinary Agent Plugins 1.0 packages and a Copilot marketplace index. Version 7 narrows the project to exactly this problem and removes the general agent-platform surfaces it had accumulated.

1. The problem

Copilot reads plain files: `SKILL.md`, custom agent profiles, `mcp.json`. Plain files are the right runtime format and a poor unit of reuse. When the same workflow is needed by many teams, the choices are to copy it, which drifts, or to write it generically, which leaves each team to supply its own facts in conversation every time.

The pain is concrete. A developer maintaining one README skill across thirty integration repositories edits thirty near-identical files. A platform team that improves a shared "self-heal" workflow cannot reach the teams that copied it. Two teams publish skills called `review` to one marketplace, and which one a person gets depends on load order. A plugin ships an MCP server named `github`, and Copilot gives the plugin's server precedence over the user's own.

2. Documents and data

TypeFence separates two things that plain files mix together. **Documents** are Markdown: instructions, guides, norms, reference material. They have no type, because typing prose protects nothing; a model accepts any text a person types. **Data** is the small set of values that differ from one team or repository to the next: a team's name, its ticket queue, its release tier.

Types apply only to data. A context type declares fields, their types, which are required, their defaults and allowed choices, and labels and help text. That declaration is a contract. Every team that instantiates a shared template provides data of the same type, the build checks every blank in every instance, and the same declaration is enough to generate an input form. When the contract gains a required field, every team's data that lacks it fails the marketplace build, by name.

3. Templates and instances

A document, skill, or profile becomes a template by declaring parameters, and uses them through field references such as `{{team.queue}}`. References insert values; there are no expressions, conditionals, or loops, so the compiler can check every one.

Templates are never shipped; instances are. An agent binds its team's data with `with`, and every template skill it composes is emitted as `<instance name>-<skill>`, with the instance name taken from the data itself. A skill can also instantiate a template directly, which is how one developer turns one README template into one concrete skill per repository. A team that needs more than different values extends the shared skill: an extension appends its own instructions to the base's, inherits its contract, and picks up every fix to the base on the next build.

4. Checks the format cannot express

The plugin format has no way to say that a skill needs a particular server, that a slash command depends on context only its agent supplies, or that a runner expects a particular result shape. TypeFference checks each at build time:

- **Servers.** A skill that requires an MCP server always ships with that server's configuration, and every server name is namespaced and denotes one configuration across the marketplace.
- **Self-contained skills.** A skill shipped on its own has no unbound parameters, so it behaves the same whether a person runs it as a slash command or an agent invokes it.
- **Output contracts.** A skill's input and output schemas ship beside it, so an automated runner validates against the same contract the instructions name.
- **Names.** Every skill, agent, plugin, and server name maps to one thing in the build.

5. One marketplace, many packages

Teams author packages in their own repositories and publish them to a feed: Azure Artifacts, JFrog Artifactory, a Git repository, or a directory. A marketplace package pins one version of each team's package and lists the plugins to ship. Because the marketplace is one build, the name rules, the one-version rule, and the compatibility report hold across every team, and each plugin carries its owning package's version and provenance. Releasing one team's package changes only that team's plugins.

Builds are deterministic. Identical source produces identical bytes on every platform, so a change to a shared template becomes a reviewable diff of exactly what changes for every team.

6. Boundaries

TypeFference authors plugins; it does not install, host, enable, authenticate, or run them. Copilot and GitHub do. It never emits a secret and never depends on environment variables in plugin configuration; MCP sign-in is the server's and Copilot's concern. Copilot-only frontmatter is emitted only when a document asks for it, so the portable Agent Plugins core stays the default and a second host would be an adapter rather than a rewrite.

Version 7 removed what no one was using: the neutral bundle, ARD and A2A publication, deployment linking, signed source packages, structural interfaces, and the behavioural-equivalence harness. Their reasoning remains in the decision records. What remains is small enough to explain in a sentence: write your organization's Copilot know-how once, specialize it per team with typed data, review it like code, and ship it everywhere through one marketplace.

References

- 1. Agent Plugins specification: <https://github.com/agentplugins/agent-plugins-spec>
- 2. GitHub Copilot CLI plugin reference: <https://docs.github.com/en/enterprise-cloud@latest/copilot/reference/copilot-cli-reference/cli-plugin-reference>
- 3. Agent Skills specification: <https://agentskills.io/specification>
- 4. Model Context Protocol: <https://modelcontextprotocol.io/specification/>